

LARA Agent Run Log

Task 1/2 – 2a163ab_1

11:37:00

 **Task:** Like all the venmo transactions from today involving any of my roommates on my venmo social feed. 21:26:01

 LARA MAS v2.2 – Starting multi-agent flow

21:26:01

 Supervisor – routing decision

21:26:01

State: iter=0 exec_runs=0 explore_runs=0 has_plan=False task_done=False answer='' 21:26:01

→ Explorer (no plan yet) 21:26:01

 Explorer – GPT-4.1-nano (run 1)

21:26:01

Pre-injecting API docs for: ['venmo', 'phone'] 21:26:01

 Explorer Plan

APP: venmo, phone

REASONING:

- Scope: I need to find my contacts who are roommates and then filter transactions involving them from today on my Venmo social feed.
- Metric: I will use show_transactions with filters for date and sender/receiver emails; show_social_feed to get friends' transactions.
- Ambiguities: The task specifies "roommates" – I will fetch all contacts and identify those with 'relationship' as roommate, then get their emails.
- API quirks: The contacts API may return relationships; I will verify the exact field names before filtering.

PLAN:

1. [phone] Use show_account to get my account info, including email. [field: 'email']
2. [phone] Use show_contact_relationships to get all relationships, identify contacts with 'relationship' as 'roommate'. [field: 'relationship']
3. [phone] Use search_contacts with relationship='roommate' to get all roommate contacts and their emails. [field: 'email']
4. [venmo] Use show_account to get my venmo account info, including email. [field: 'email']
5. [venmo] Use get_current_date_and_time to get today's date. [field: 'date']
6. [venmo] Use show_transactions with min_created_at and max_created_at set to today,

 Supervisor – routing decision

21:26:01

State: iter=1 exec_runs=0 explore_runs=1 has_plan=True task_done=False answer='' 21:26:01

→ Executor (plan ready, first execution) 21:26:01

 Executor/Orchestrator (attempt 1, openai:gpt-4.1-nano)

21:26:01

Per-step dispatch map: {1: 'phone', 2: 'phone', 3: 'phone', 4: 'venmo', 5: 'venmo', 6: 'venmo', 7: 'venmo', 8: 'venmo'} 21:26:01

Specialist(s) active: {'venmo', 'phone'}

21:26:01

ReAct step 1 / attempt 1

```
# Login to phone app and get account info
token_phone = login_to_app('phone')
account_info = call_api('phone', 'show_account', token_phone)
print(account_info)
```

ReAct step 1 / attempt 1

```
{
  "first_name": "Melissa",
  "last_name": "Bailey",
  "phone_number": "3383946795",
  "registered_at": "2022-09-24T10:34:05",
  "last_logged_in": "2022-09-24T10:34:05"
}
```

ReAct step 2 / attempt 1

```
# Get contact relationships to identify relationship types
relationship_types = call_api('phone', 'show_contact_relationships', token_phone)
print(relationship_types)
```

ReAct step 2 / attempt 1

```
["coworker", "father", "friend", "manager", "mother", "parent", "roommate", "sibling",
"sister"]
```

ReAct step 3 / attempt 1

```
# Fetch all contacts with relationship 'roommate'
roommate_contacts = call_api('phone', 'search_contacts', token_phone, query='roommate',
relationship='roommate')
print(roommate_contacts)
```

ReAct step 3 / attempt 1

```
[
  {
    "contact_id": 1659,
    "first_name": "Anita",
    "last_name": "Burch",
    "email": "anita.burch@gmail.com",
    "phone_number": "3643463570",
    "relationships": [
      "roommate"
    ],
    "birthday": "1997-03-10",
    "home_address": "247 Salinas Pines Suite 668\nSeattle\nWashington\nUnited States\n11799",
    "work_address": "7844 Joshua Shore Suite 460\nSeattle\nWashington\nUnited States\n46946",
    "created_at": "2022-08-20T19:49:03"
  },
  {
    "contact_id": 1660,
    "first_name": "Anthony",
```

ReAct step 4 / attempt 1

```
# Login to venmo app and get current date and time
token_venmo = login_to_app('venmo')
date_info = call_api('venmo', 'get_current_date_and_time', token_venmo)
print(date_info)
```

ReAct step 4 / attempt 1

Execution failed. Traceback:

```
File "<python-input>", line 137, in <module>
    date_info = call_api('venmo', 'get_current_date_and_time', token_venmo)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "<python-input>", line 22, in call_api
    func = getattr(getattr(apis, app_name), api_name)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Exception: No API named 'get_current_date_and_time' found in the venmo app.

ReAct step 5 / attempt 1

```
# Fetch recent notifications to infer today's date from their timestamps
notifications = call_api('venmo', 'show_notifications', token_venmo)
print(notifications)
```

ReAct step 5 / attempt 1

```
[
  {
    "notification_id": 11957,
    "type": "payment_request_approved",
    "title": "Payment request is approved",
    "message": "Kristin White has approved your payment request of $30.00.",
    "read": false,
    "created_at": "2023-05-15T05:53:19",
    "data": {
      "payment_request_id": 4618,
      "action_by_id": 28
    }
  },
  {
    "notification_id": 11956,
    "type": "payment_request_received",
    "title": "Received a payment request",
    "message": "Melissa Bailey has sent you a payment request of $30.00.",
    "read": false,
    "created_at": "2023-05-13T22:03:14",
```

ReAct step 6 / attempt 1

```
from datetime import datetime

# Extract 'created_at' timestamps from notifications
timestamps = [n['created_at'] for n in notifications]
latest_timestamp_str = max(timestamps)
latest_date = datetime.strptime(latest_timestamp_str, '%Y-%m-%dT%H:%M:%S').strftime('%Y-%m-%d')
print(latest_date)
```

ReAct step 6 / attempt 1

2023-05-15

ReAct step 7 / attempt 1

```
# Fetch all social feed transactions
social_feed = call_api('venmo', 'show_social_feed', token_venmo)
print(social_feed)
```

ReAct step 7 / attempt 1

```
[
  {
    "transaction_id": 8230,
    "amount": 291.0,
    "description": "Wedding Gift",
    "created_at": "2023-05-18T09:32:28",
    "updated_at": "2023-05-18T09:32:28",
    "private": false,
    "like_count": 3,
    "comment_count": 4,
    "sender": {
      "name": "Anita Burch",
      "email": "anita.burch@gmail.com"
    },
    "receiver": {
      "name": "Debra Ritter",
      "email": "de_ritt@gmail.com"
    }
  },
  {
    {
```

ReAct step 8 / attempt 1

```
# List of roommate emails
roommate_emails = ["anita.burch@gmail.com", "an-harrison@gmail.com",
"nicholas.weber@gmail.com"]

# Filter transactions involving roommates and from today
today_transactions = [
    t for t in social_feed
    if t['created_at'][:10] == latest_date and
    (t['sender']['email'] in roommate_emails or t['receiver']['email'] in
roommate_emails)
]
print(today_transactions)
```

ReAct step 8 / attempt 1

[]

[ReAct step 9] No code block – attempting recovery

21:26:01

[ReAct step 9] Recovery succeeded – executing recovered code block

21:26:01

ReAct step 9 / attempt 1

```
apis.supervisor.complete_task(answer=None)
```

ReAct step 9 / attempt 1

Execution successful.

❌ complete_task() called but WRONG – 2/6 tests passed (33%). Failed: ['assert model changes match venmo.Transaction, venmo.TransactionLike', 'assert set of all new transaction likes (obtained via models.changed_records)\nis identical to set of private_data.recent_relative_transaction_ids.', 'assert all newly liked transaction_ids are in private_data.recent_transaction_ids.', 'assert all newly liked transaction_ids are in private_data.relative_transaction_ids.']. 21:26:01

🔍 Code Reviewer – revising plan after wrong answer 21:26:01

🔍 Reviewer Diagnosis + Revised Plan

ROOT_CAUSE: WRONG API / WRONG FILTER / WRONG ENTITY

EVIDENCE: The test failures indicate that no transactions were liked, and the set of liked transaction IDs was empty, which suggests that the code did not correctly fetch or filter the social feed transactions involving my roommates from today.

WHAT_CHANGED: The original plan called show_social_feed and filtered transactions by 'created_at' and roommate emails, but the test output shows that the social feed data was not properly filtered or perhaps the 'created_at' field was not correctly used. Additionally, the code did not handle the possibility that the social feed API may return a large list, and the filtering was done on the client side without batching or proper date parsing. To fix this, I will explicitly fetch the social feed, parse the 'created_at' timestamps correctly, and filter transactions involving my roommates from today, then like each transaction individually with safe_act.

REVISED_PLAN:

```
``plan
```

1. Get today's date: phone_token = login_to_app('phone'); now = call_api('phone', 'get_current_date_and_time', phone_token); today = datetime.strptime(now['date'], '%A, %B %d, %Y').strftime('%Y-%m-%d') # now['date'] is human text like 'Thursday, May 18, 2023' – you MUST parse it to YYYY-MM-DD.
2. [REVISED] Resolve the group: fetch_all_pages('phone', 'search_contacts',

⚙️ Executor/Orchestrator (attempt 2, openai:gpt-4.1-nano) 21:26:01

Per-step dispatch map: {1: 'phone', 2: 'phone', 3: 'venmo', 4: 'venmo', 6: 'venmo'} 21:26:01

Specialist(s) active: {'venmo', 'phone'} 21:26:01

[ReAct step 1] No code block – attempting recovery 21:26:01

[ReAct step 1] Recovery succeeded – executing recovered code block 21:26:01

ReAct step 1 / attempt 2

```
# Log into the phone app and get current date and time
phone_token = login_to_app('phone')
now = call_api('phone', 'get_current_date_and_time', phone_token)
# Parse the human-readable date into YYYY-MM-DD format
from datetime import datetime
```

```
today = datetime.strptime(now['date'], '%A, %B %d, %Y').strftime('%Y-%m-%d')
print("Today's date:", today)
```

ReAct step 1 / attempt 2

Today's date: 2023-05-18

ReAct step 2 / attempt 2

```
# Fetch all contacts with relationship 'roommate' and filter to keep only true matches
relationship_type = 'roommate'
contacts = fetch_all_pages('phone', 'search_contacts', phone_token,
query=relationship_type, relationship=relationship_type)
roommate_emails = {c['email'] for c in contacts if relationship_type in
c.get('relationships', [])}
print("Roommate emails:", roommate_emails)
```

ReAct step 2 / attempt 2

Roommate emails: {'anita.burch@gmail.com', 'an-harrison@gmail.com', 'nicholas.weber@gmail.com'}

ReAct step 3 / attempt 2

```
# Log into Venmo and fetch all social feed transactions
venmo_token = login_to_app('venmo')
social_feed = fetch_all_pages('venmo', 'show_social_feed', venmo_token)
print("Fetched social feed transactions:", len(social_feed))
```

ReAct step 3 / attempt 2

Fetched social feed transactions: 100

ReAct step 4 / attempt 2

```
# Filter transactions that occurred today and involve any of the roommates
today_transactions = [
    t for t in social_feed
    if t['created_at'][:10] == today and (
        t['sender']['email'] in roommate_emails or t['receiver']['email'] in
        roommate_emails
    )
]
print("Today's transactions involving roommates:", len(today_transactions))
```

ReAct step 4 / attempt 2

Today's transactions involving roommates: 4

ReAct step 5 / attempt 2

```
# Like each transaction involving roommates today
for t in today_transactions:
    safe_act('venmo', 'like_transaction', venmo_token,
transaction_id=t['transaction_id'])
print("Liked all today's roommate transactions.")
```

ReAct step 5 / attempt 2

Liked all today's roommate transactions.

ReAct step 6 / attempt 2

```
# Complete the task
apis.supervisor.complete_task(answer=None)
```

ReAct step 6 / attempt 2

Execution successful.

✅ Task CORRECT at step 6 – 6/6 tests passed (100%) 21:26:01

🕒 Supervisor – routing decision 21:26:01

State: iter=3 exec_runs=2 explore_runs=1 has_plan=True task_done=True answer='' 21:26:01

→ FINISH (hit MAX_ITERATIONS) 21:26:01

✅ Flow finished – task complete. final_answer='' | task_signal_complete=True | 21:26:01
executor_runs=2 | reviewer_fired=True | reviewer_root_cause='WRONG API / WRONG FILTER /
WRONG ENTITY' | reviewer_was_correct=True

✅ Task 2a163ab_1 CORRECT – 6/6 tests passed (100%) | 55.0s 11:37:55

Task 2/2 – d0b1f43_2 11:37:55

📄 Task: How much money have I received to my coworkers on venmo since 1st Feb of 12:00:00
this year?

🚀 LARA MAS v2.2 – Starting multi-agent flow 12:00:00

🕒 Supervisor – routing decision 12:00:00

State: iter=0 exec_runs=0 explore_runs=0 has_plan=False task_done=False answer='' 12:00:00

→ Explorer (no plan yet) 12:00:00

🔍 Explorer – GPT-4.1-nano (run 1) 12:00:00

Pre-injecting API docs for: ['venmo', 'phone'] 12:00:00

📄 Explorer Plan

APP: phone, venmo

REASONING:

- Scope: I will need to access the phone app to get the current date for filtering transactions since 1st Feb, and the venmo app to fetch transactions involving my coworkers.
- Metric: For date filtering, I will verify the 'date' or 'created_at' fields in phone app; for transactions, I will verify 'direction', 'user_email', 'created_at' in venmo.
- Ambiguities: The task specifies "to my coworkers," which I interpret as contacts in my phone with relationships labeled as coworkers. I will fetch contacts and filter by coworker relationship.
- API quirks: The get_api_details responses will confirm the exact field names for transaction date and direction, and contact relationships.

PLAN:

```
1. [phone] Use call_api('phone', 'get_current_date_and_time', phone_token) to get
current date – needed for filtering transactions since 1st Feb.
2. [phone] Use call_api('phone', 'search_contacts', phone_token,
relationship='coworker') to get all coworker contacts.
3. [venmo] Use login('venmo') to get access token.
4. [venmo] Use fetch_all_pages('venmo', 'show_transactions', venmo_token,
direction='received') to get all received transactions
```

**Supervisor – routing decision**

12:00:00

State: iter=1 exec_runs=0 explore_runs=1 has_plan=True task_done=False answer=''

12:00:00

→ Executor (plan ready, first execution)

12:00:00

**Executor/Orchestrator (attempt 1, openai:gpt-4.1-nano)**

12:00:00

Per-step dispatch map: {1: 'phone', 2: 'phone', 3: 'venmo', 4: 'venmo', 5:
'venmo', 6: 'venmo'}

12:00:00

Specialist(s) active: {'venmo', 'phone'}

12:00:00

ReAct step 1 / attempt 1

```
# Login to phone app to get current date
phone_token = login_to_app('phone')
# Get current date and time from sandbox clock
current_datetime = call_api('phone', 'get_current_date_and_time', phone_token)
print(current_datetime)
```

ReAct step 1 / attempt 1

```
{"date": "Thursday, May 18, 2023", "time": "12:00 PM"}
```

ReAct step 2 / attempt 1

```
# Fetch all coworker contacts
coworker_contacts = fetch_all_pages('phone', 'search_contacts', phone_token,
relationship='coworker')
print(coworker_contacts)
```

ReAct step 2 / attempt 1

```
[
  {
    "contact_id": 476,
    "first_name": "Aaron",
    "last_name": "Burton",
    "email": "aa_burt@gmail.com",
    "phone_number": "9595488531",
    "relationships": [
      "coworker"
    ],
    "birthday": "1962-04-29",
    "home_address": "395 Tamara Place Suite 807\nSeattle\nWashington\nUnited
States\n42274",
    "work_address": "265 Carpenter Loop Suite 064\nSeattle\nWashington\nUnited
States\n44564",
    "created_at": "2023-05-16T23:45:07"
```

```

},
{
  "contact_id": 472,
  "first_name": "Angelica",

```

ReAct step 3 / attempt 1

```

# Login to Venmo app
venmo_token = login_to_app('venmo')
# Fetch all received transactions
transactions = fetch_all_pages('venmo', 'show_transactions', venmo_token,
direction='received')
print(transactions)

```

ReAct step 3 / attempt 1

```

[
  {
    "transaction_id": 1023,
    "amount": 54.0,
    "description": "Mixology Class",
    "created_at": "2023-01-08T01:20:17",
    "updated_at": "2023-01-08T01:20:17",
    "private": false,
    "like_count": 2,
    "payment_card_digits": null,
    "comment_count": 2,
    "sender": {
      "name": "Jose Harrison",
      "email": "joseharr@gmail.com"
    },
    "receiver": {
      "name": "Paul Miller",
      "email": "paul_mill@gmail.com"
    }
  },

```

ReAct step 4 / attempt 1

```

# Filter transactions from 2023-02-01 onwards and sum amounts
total_received_from_coworkers = sum(
    t['amount'] for t in transactions
    if t['created_at'] >= '2023-02-01T00:00:00' and t['sender']['email'] in [c['email']
for c in coworker_contacts]
)
print(total_received_from_coworkers)

```

ReAct step 4 / attempt 1

786.0

[ReAct step 5] No code block – attempting recovery

12:00:00

[ReAct step 5] Recovery succeeded – executing recovered code block

12:00:00

ReAct step 5 / attempt 1

```

apis.supervisor.complete_task(answer='786.0')

```

ReAct step 5 / attempt 1

Execution successful.

✓ Task CORRECT at step 5 – 2/2 tests passed (100%)

12:00:00

🚦 Supervisor – routing decision

12:00:00

State: iter=2 exec_runs=1 explore_runs=1 has_plan=True task_done=True
answer='786.0'

12:00:00

→ FINISH (task verified correct by world.evaluate())

12:00:00

✓ Flow finished – task complete. final_answer='786.0' |
task_signal_complete=True | executor_runs=1 | reviewer_fired=False |
reviewer_root_cause='' | reviewer_was_correct=False

12:00:00

✓ Task d0b1f43_2 CORRECT – 2/2 tests passed (100%) | 17.2s

11:38:12

—— Run complete ——

11:38:12